

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет Компьютерных наук
Кафедра программирования и информационных технологий

«Приложение для создания уникальных обложек для музыкальных
произведений VisualMusic»

Курсовая работа
Направление: 09.03.02 Информационные системы и технологии

Зав. Кафедрой	_____	д. ф.-м. н, доцент С.Д. Махортов
Руководитель	_____	ст. преподаватель В.С. Тарасов
Руководитель практики		ассистент, Ю.В. Шишко
Обучающийся		Е.А. Белых, 3 курс, д/о
Обучающийся		А.О. Родионов, 3 курс, д/о
Обучающийся		А.В. Липовцев, 3 курс, д/о
Обучающийся		И.С. Бажанов, 3 курс, д/о
Обучающийся		Р.Г. Сейдалиев, 3 курс, д/о
Обучающийся		Г.Е. Тамбовцев, 3 курс, д/о

Воронеж 2025

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
Введение.....	4
1 Постановка задачи.....	5
1.1 Требования к разрабатываемой системе	5
1.1.1 Функциональные требования	5
1.1.2 Нефункциональные требования	10
1.2 Требования к архитектуре	12
2 Анализ предметной области	14
2.1 Термины и сокращения.....	14
2.2 Обзор аналогов.....	15
2.2.1 VEED	15
2.2.2 Fotor	16
2.2.3 Night Cafe Creator	16
2.2.4 Вывод по обзору аналогов	17
2.3 Диаграммы, иллюстрирующие работу системы	18
2.3.1 Диаграмма прецедентов	18
2.3.2 Диаграмма последовательности	20
2.3.3 Диаграмма состояний	24
2.3.4 Диаграмма активностей.....	25
2.3.5 ER-диаграмма	27
2.3.6 Диаграмма развертывания	27
3 Реализация.....	29
3.1 Средства реализации	29
3.1.1 Средства реализации серверной части экрана.....	29

3.1.2 Средства реализации клиентской части сайта	29
3.1.3 Средства реализации нейросетевой части сервиса	30
3.2 Реализация серверной части приложения	30
3.3 Реализация админ-панели	32
3.4 Реализация нейросетевых модулей	36
3.5 Клиентская часть приложения	44
3.5.1 Неавторизованный пользователь	44
3.5.2 Авторизованный пользователь	45
Заключение	51
Список использованных источников	52

Введение

В современном мире музыка играет важную роль не только как форма искусства, но и как средство самовыражения, коммуникации и создания атмосферы.

С развитием цифровой культуры и ростом популярности платформ для прослушивания треков возникает потребность в более глубоком и эмоционально точном визуальном сопровождении музыкального контента.

Обложка трека — это первое, с чем сталкивается слушатель. Она формирует ожидания, передаёт настроение композиции и помогает выделиться среди множества других записей. Однако создание визуального оформления требует, как времени, так и художественных навыков, что делает этот процесс недоступным для многих исполнителей и авторов.

В этой связи актуальным становится подход, при котором визуальное оформление создается автоматически на основе содержания самой композиции. Такой подход позволяет сделать творчество более доступным и демократичным, открывая новые формы взаимодействия между аудиальной и визуальной составляющими.

Проект, посвящённый автоматической генерации обложек, ставит своей целью упростить и переосмыслить процесс визуального представления музыки, основываясь на её жанре, тексте и эмоциональной окраске. Это позволяет каждому треку обрести собственный уникальный визуальный образ, более точно отражающий его содержание и настроение.

1 Постановка задачи

Целями создания приложения являются:

- реализация системы, которая автоматизирует процесс создания уникальных обложек для музыкальных произведений, сокращая время и повышая удобство подготовки визуального контента для исполнителей, продюсеров и независимых артистов;
- разработать алгоритмы, способные создавать обложки, отражающие музыкальную идентичность трека, что позволит пользователям выделиться на стриминговых платформах за счет уникального дизайна;
- внедрение премиум-подписки с такими функциями, как неограниченное количество генераций и доступ к расширенным возможностям редактора изображений, с целью увеличения вовлеченности пользователей и обеспечения стабильного дохода заказчику.

1.1 Требования к разрабатываемой системе

1.1.1 Функциональные требования

Разрабатываемое приложение должно предоставлять ряд функций для удовлетворения потребностей пользователя.

Следующие функциональные возможности приложения доступны неавторизованным пользователям.

Пользователю предоставляются следующие возможности для авторизации в приложении:

- через почту и пароль;
- через сторонний сервис ВКонтакте.

Если пользователь впервые в приложении, то имеется возможность регистрации через почту и пароль.

Пользователь имеет возможность просматривать работы других пользователей. При необходимости возможна фильтрация работ по следующим критериям:

- по жанрам;
- по настроению;
- по стилистике изображения.

Также возможна сортировка работ следующими способами:

- по популярности;
- по дате создания.

Пользователь имеет следующие возможности для взаимодействия с обложками других пользователей:

- сохранить изображение на устройство;
- открыть изображение в встроенном редакторе изображений.

Неавторизованный пользователь имеет возможность применять фильтры к исходному изображению в редакторе изображений.

Следующие функциональные возможности предоставляются авторизованному пользователю.

Авторизованный пользователь без подписки может сгенерировать до трех обложек в день. Если у пользователя есть подписка, то ему доступно неограниченное число генераций. Для генерации доступны следующие режимы:

- автоматический режим;
- режим детальной настройки;

- генерация по конкретному запросу.

Для генерации в автоматическом режиме необходимо предоставить аудиофайл в формате WAV.

Для генерации в режиме детальной настройки пользователь может предоставить аудиофайл в формате WAV и при желании дополнительно настроить следующие параметры:

- жанр музыкального произведения;
- настроение музыкального произведения;
- стиль изображения;
- текст музыкального произведения.

Либо пользователь может не предоставлять аудиофайл и указать следующий необходимый минимум параметров для генерации изображения:

- жанр;
- настроение;
- текст.

Для генерации в режиме конкретного запроса, пользователю необходимо ввести текстовое описание того, что он желает видеть на обложке.

После ввода всех необходимых данных в любом из режимов генерации, пользователю необходимо будет указать название обложки.

Авторизированный пользователь имеет возможность просматривать коллекцию:

- созданных обложек;
- сохраненных обложек.

Авторизированный пользователь имеет следующие возможности взаимодействия с обложками других пользователей:

- добавить в сохраненные;
- открыть в редакторе изображений;
- сохранить на устройство.

Для взаимодействия с созданной обложкой возможны следующие действия:

- сделать публичной или приватной (видимой или невидимой для других пользователей);
- удалить из коллекции;
- сгенерировать заново с заданными параметрами генерации;
- открыть в редакторе изображений;
- сохранить на устройство.

Авторизированному пользователю без подписки доступна только возможность применения фильтра к изображению. Если он имеет подписку, то варианты взаимодействия дополняются следующими:

- добавить текст;
- сделать разметку изображения;
- добавить фото поверх исходного изображения.

Следующие функциональные возможности предоставляются администратору.

Администратор имеет следующие возможности взаимодействия с пользователями приложения:

- просмотреть список всех пользователей;

- просмотреть данные пользователя;
- удалить пользователя.

Администратор имеет следующие возможности взаимодействия с работами других пользователей:

- просмотреть список работ всех пользователей (публичных и частных);
- просмотреть данные любой обложки;
- удалить обложку.

1.1.2 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы, которые не связаны напрямую с функциональностью, но влияют на общее качество и удобство использования приложения. Ниже приведены нефункциональные требования для разрабатываемого приложения.

Время отклика:

- время обработки фотографий не должно превышать 5 минут.
- время загрузки фотографий в галерею — не более 5 секунд.
- время отклика сервера на запросы — не более 1000 мс.

Оптимизация ресурсов:

- минимизация использования оперативной памяти и процессора на устройствах пользователей.
- оптимизация загрузки изображений для уменьшения потребления трафика.

Стабильность работы:

- приложение должно работать без сбоев 95% времени;
- время восстановления после сбоя не должно превышать 60 минут.

Обработка ошибок:

- приложение должно корректно обрабатывать ошибки и предоставлять пользователю понятные сообщения об ошибках.

Защита данных:

- использование JWT-токенов для авторизации пользователей с ограниченным временем жизни (5-10 минут);
- защита от SQL-инъекций;

- шифрование персональных данных пользователей с использованием современных алгоритмов шифрования (Vcrypt, SHA-256 или AES-256);
- шифрование данных при передаче между клиентом и сервером с использованием HTTPS.

Конфиденциальность:

- возможность настройки приватности созданных обложек.

Масштабируемость:

- приложение должно поддерживать до 25 одновременных пользователей;
- возможность масштабирования системы для поддержки большего количества пользователей в будущем.

Поддержка различных устройств:

- приложение должно корректно работать на устройствах с объемом оперативной памяти от 4 ГБ;
- поддержка устройств с различной производительностью процессора.

Энергопотребление:

- приложение должно минимизировать потребление энергии на мобильных устройствах.

1.2 Требования к архитектуре

Приложение должно разрабатываться на основе модели клиент-серверного взаимодействия, использующей REST API для обмена данными между клиентом и сервером, а также между сервером и нейросетевыми микросервисами.

Мобильная часть приложения должна быть реализована в соответствии с архитектурным шаблоном MVVM и архитектурным подходом Clean Architecture.

Шаблон MVVM позволяет четко разделять бизнес-логику и логику представления, что упрощает тестирование, обслуживание и развитие приложения.

Архитектурный подход Clean Architecture предоставляет четкое разделение приложения на модули, что упрощает сопровождение кода, добавление новых функций и тестирование.

Серверная часть должна быть реализована в соответствии с архитектурным подходом Clean Architecture.

Для реализации серверной части приложения будут использоваться следующие средства:

- язык программирования Java 17;
- среда разработки IntelliJ IDEA Ultimate 2024.3.5;
- фреймворк Spring версии 3.4.3.

Для реализации хранения данных приложения будет использовано:

- СУБД PostgreSQL;
- Объектное хранилище MinIO.

В качестве платежной системы будет использоваться:

- ЮMoney.

Для развертывания приложения будут использоваться следующие средства:

- Docker;
- VDS сервер на Aeza.

Для реализации нейросетевой и браузерной части приложения будут использоваться следующие средства:

- язык программирования Python 3.13;
- среда разработки PyCharm Professional 2024.3.5
- фреймворк Django версии 5.1.7.

Для реализации мобильного приложения будут использоваться следующие средства:

- язык программирования Kotlin 2.1.0;
- среда разработки Android Studio Meerkat 2024.3.1.
- библиотека Retrofit версии 1.9.0;
- библиотека Hilt версии 2.55.

2 Анализ предметной области

2.1 Термины и сокращения

В настоящей работе используются следующие термины и сокращения с соответствующими определениями:

- **Клиентская часть** – это совокупность программного обеспечения и интерфейса, с помощью которых пользователь взаимодействует с системой;
- **Серверная часть** – это совокупность программного обеспечения и инфраструктуры на сервере, которые обрабатывают запросы от клиентской части, управляют данными и обеспечивают работу приложения;
- **Сервер** – это устройство, в частности компьютер, которое отвечает за предоставление услуг, программ и данных другим клиентам посредством использования сети;
- **Нейросеть** – это модель машинного обучения, имитирующая работу нейронов, которая предназначена для обработки, анализа и интерпретации сложных данных посредством адаптивного обучения;
- **Фильтр** – это операция, имеющую своим результатом изменения характеристик и параметров изображения, получаемое из исходного по некоторым правилам;
- **Деплой** – это размещение готовой версии программного обеспечения на платформе, доступной для пользователей;
- **API** – это набор способов и правил, по которым различные программы общаются между собой и обмениваются данными;
- **JWT-токен** – это открытый стандарт для создания токенов доступа, основанный на формате JSON.

2.2 Обзор аналогов

Среди конкурентов сервиса для создания уникальных обложек для музыкальных произведений были выявлены следующие: VEED, Fotor и Night Cafe Creator. Данные сервисы являются косвенными конкурентами, так как несмотря на схожие с нашим приложением возможности, ориентированы на широкий спектр пользователей из-за возможности генерации только по текстовому запросу и с любым форматом изображения. Наше же приложение предназначено для пользователей, которые специализируются на создании музыкального контента.

Потребности целевой аудитории:

- создание уникальных обложек без наличия профессиональных дизайнерских навыков;
- ускорение процесса подготовки визуального контента для релизов, плейлистов или публикаций, с целью экономии бюджета и времени;
- получение мотивации для регулярного использования приложения через интерактивные функции;
- приложение должно быть простым в использовании и освоении, визуально привлекательным и доступным.

2.2.1 VEED

VEED – онлайн-платформа для редактирования изображения, работы с видео и обработки аудио.

Сильные стороны:

- встроенный редактор изображений;
- простой в освоении интерфейс;
- возможность выбрать разрешение генерируемого изображения.

Слабые стороны:

- низкое качество детализации у сгенерированных изображений;
- добавление водяного знака для сгенерированных изображений в бесплатной подписке.

2.2.2 Fotor

Fotor – универсальный онлайн-редактор фотографий и инструмент для дизайна.

Сильные стороны:

- встроенный редактор изображений;
- большое количество стилей для генерации изображений;
- возможность генерации нескольких изображений одновременно.

Слабые стороны:

- количество генераций по бесплатной подписке сильно ограничено;
- добавление водяного знака для сгенерированных изображений в бесплатной подписке.

2.2.3 Night Cafe Creator

Night Cafe Creator – платформа для генерации изображений.

Сильные стороны:

- встроенный редактор изображений;
- большое количество стилей для генерации;
- возможность генерации нескольких изображений одновременно;

- возможность выбора модели для генерации изображений.

Слабые стороны:

- бесплатная подписка сильно ограничивает функциональность приложения;
- добавление водяного знака для сгенерированных изображений в бесплатной подписке.

2.2.4 Вывод по обзору аналогов

Все три сервиса обходят стороной ключевую возможность, которую мы разрабатываем: генерация обложки по аудио, с глубоким анализом музыки и соответствием жанру, настроению, и требованиям музыкальных платформ.

Наше приложение:

- Закрывает нишевую потребность артистов.
- Предлагает уникальный UX для музыкантов.
- Объединяет визуальное искусство и звук — то, чего сегодня нет у конкурентов.

2.3 Диаграммы, иллюстрирующие работу системы

2.3.1 Диаграмма прецедентов

Диаграмма прецедентов (Use Case Diagram) – диаграмма, описывающая, какой функционал разрабатываемой программной системы доступен каждой группе пользователей. На Рисунках 1-4 представлены диаграммы прецедентов для неавторизованных пользователей, авторизованных пользователей без подписки, авторизованных пользователей с подпиской и администратора.

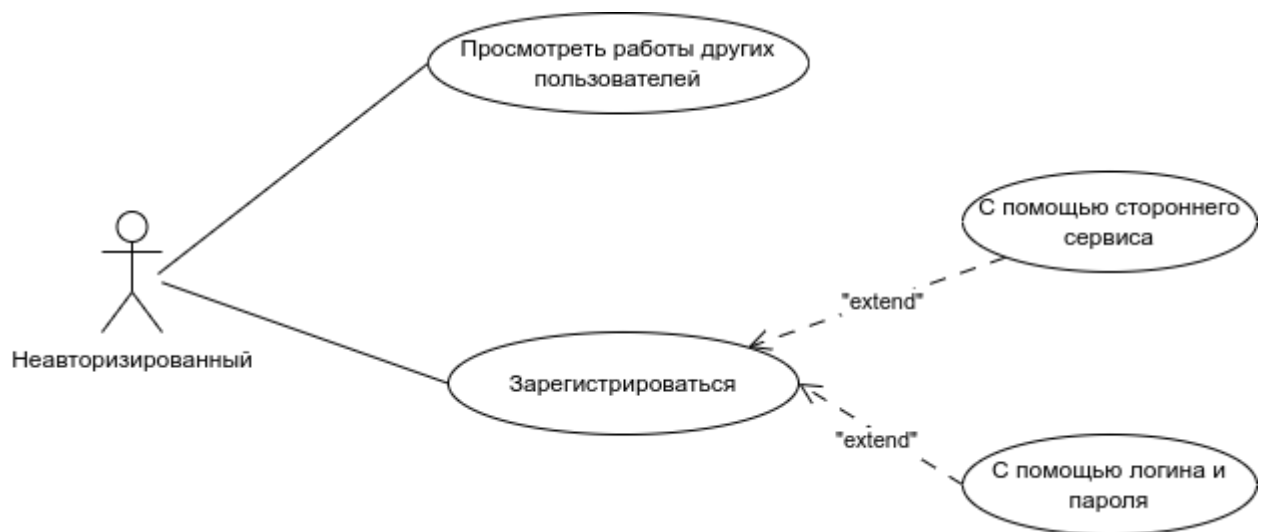


Рисунок 1 - Use Case Diagram. Неавторизованный пользователь

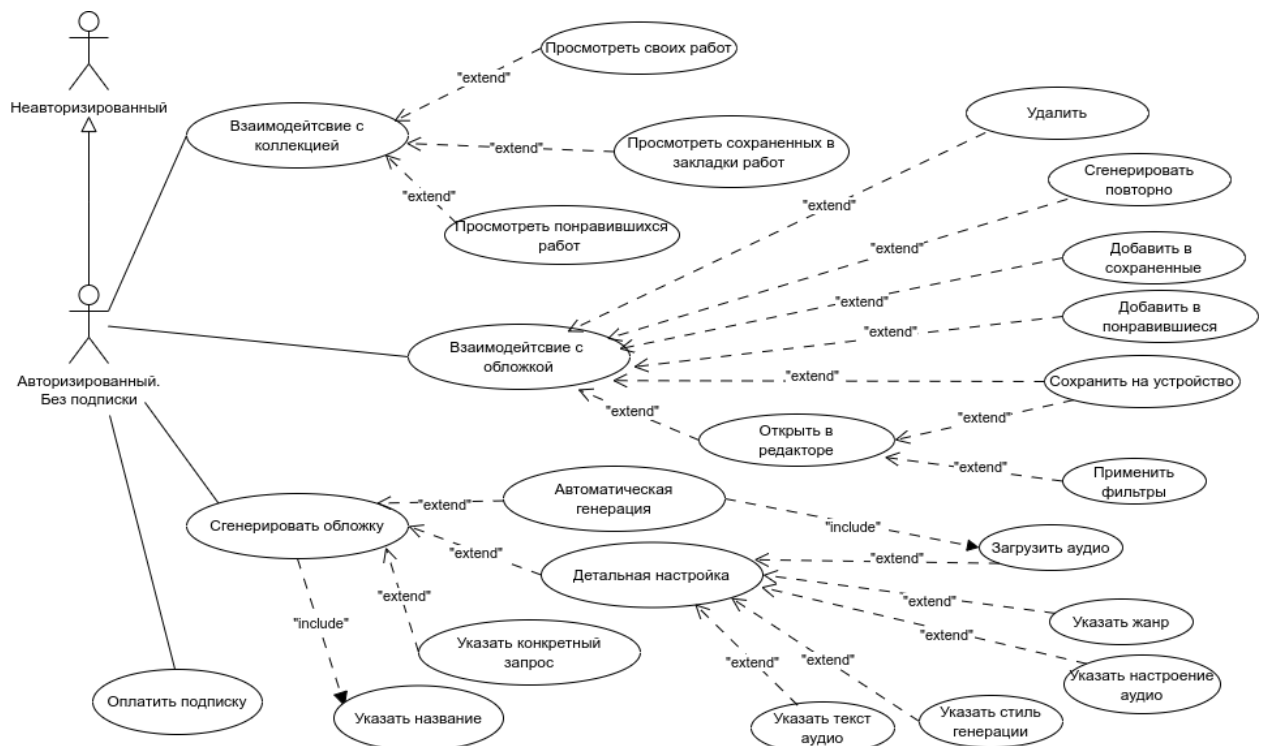


Рисунок 2 - Use Case Diagram. Авторизованный пользователь без подписки

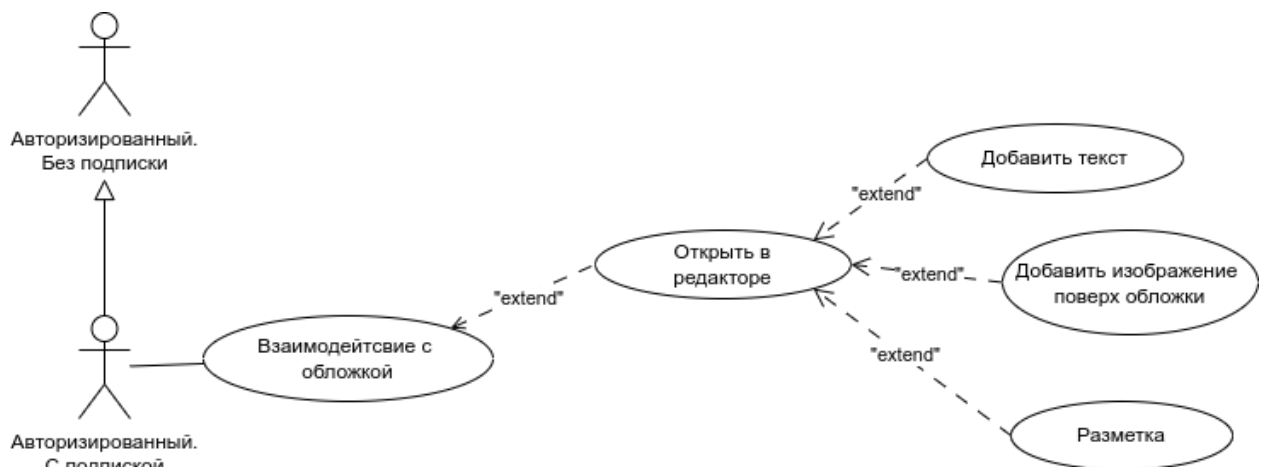


Рисунок 3 - Use Case Diagram. Авторизованный пользователь с подпиской

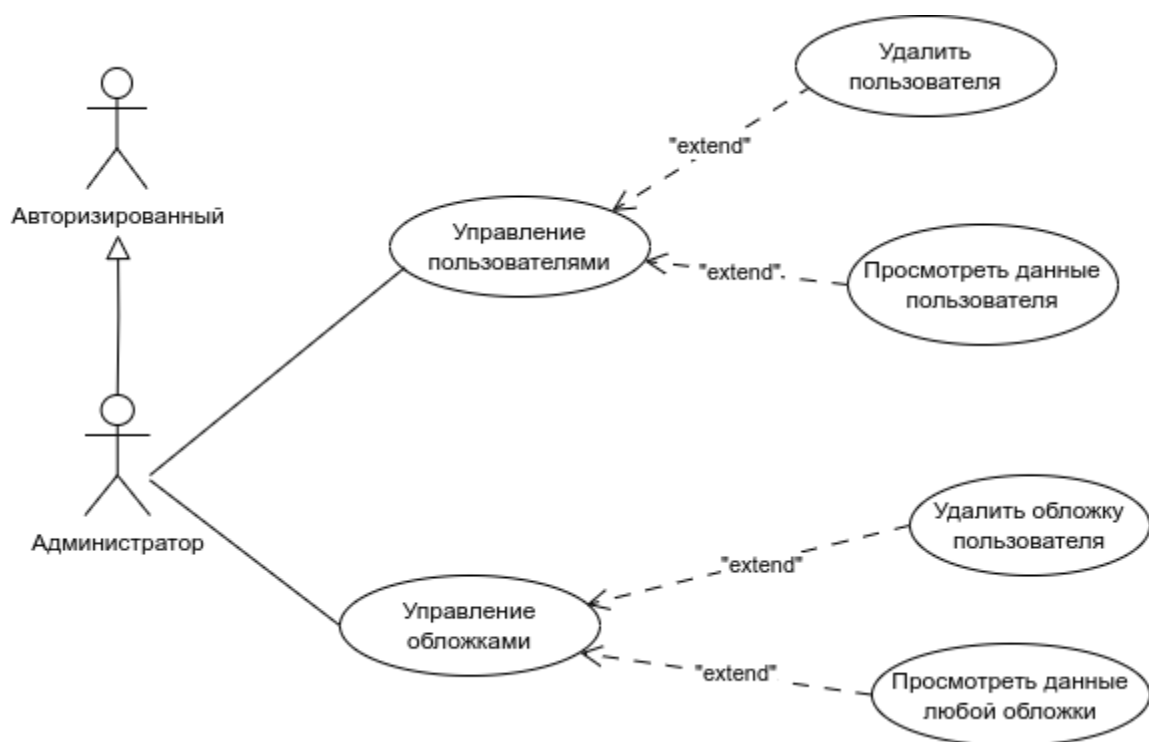


Рисунок 4 - Use Case Diagram. Администратор

2.3.2 Диаграмма последовательности

На диаграмме последовательности (Sequence diagram) для некоторого набора объектов на единой временной оси показан жизненный цикл объекта и взаимодействие актеров информационной системы в рамках прецедента. На Рисунках 5-7 представлены диаграммы последовательности для генерации пользователем обложки и для ее редактирования.

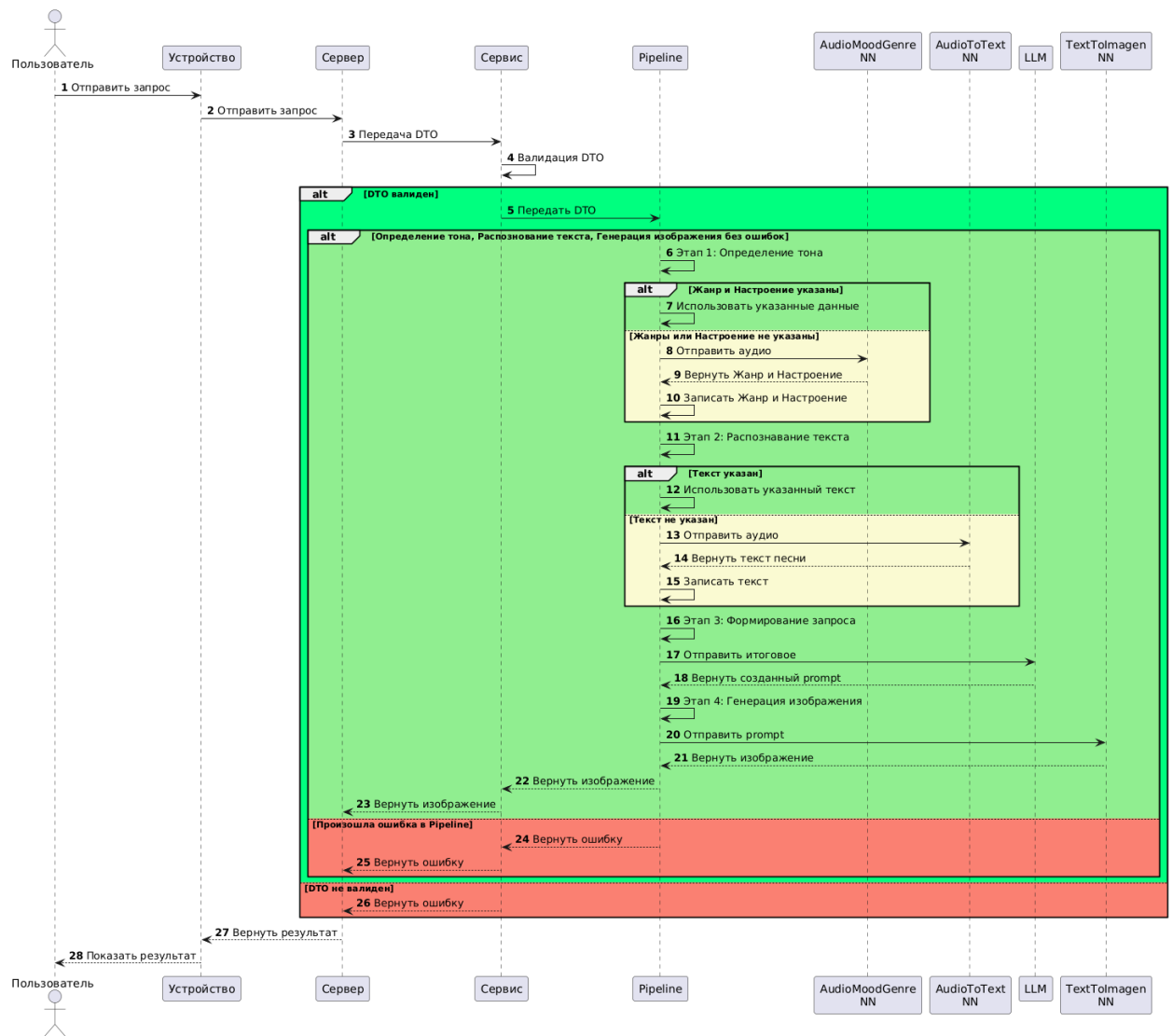


Рисунок 5 - Sequence Diagram. Генерация обложки в автоматическом или детальном режимах

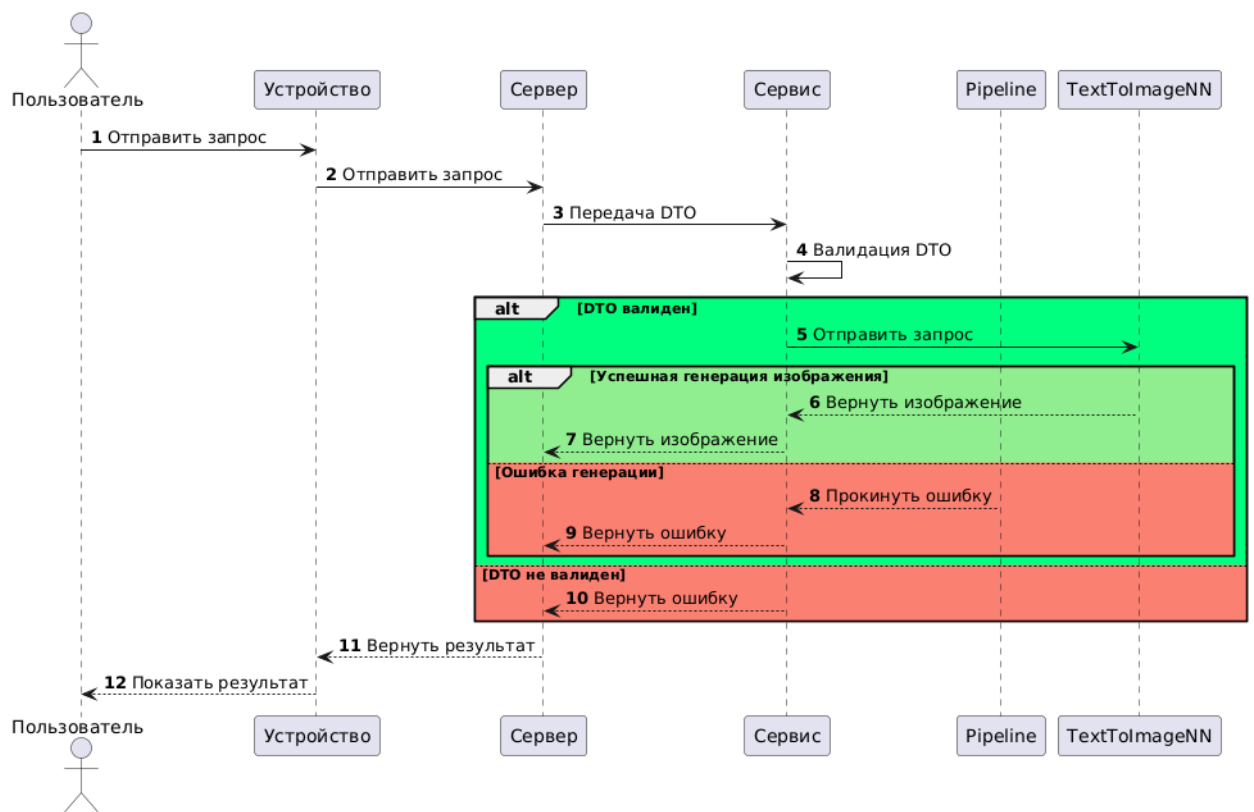


Рисунок 6 - Sequence Diagram. Генерация обложки по конкретному запросу

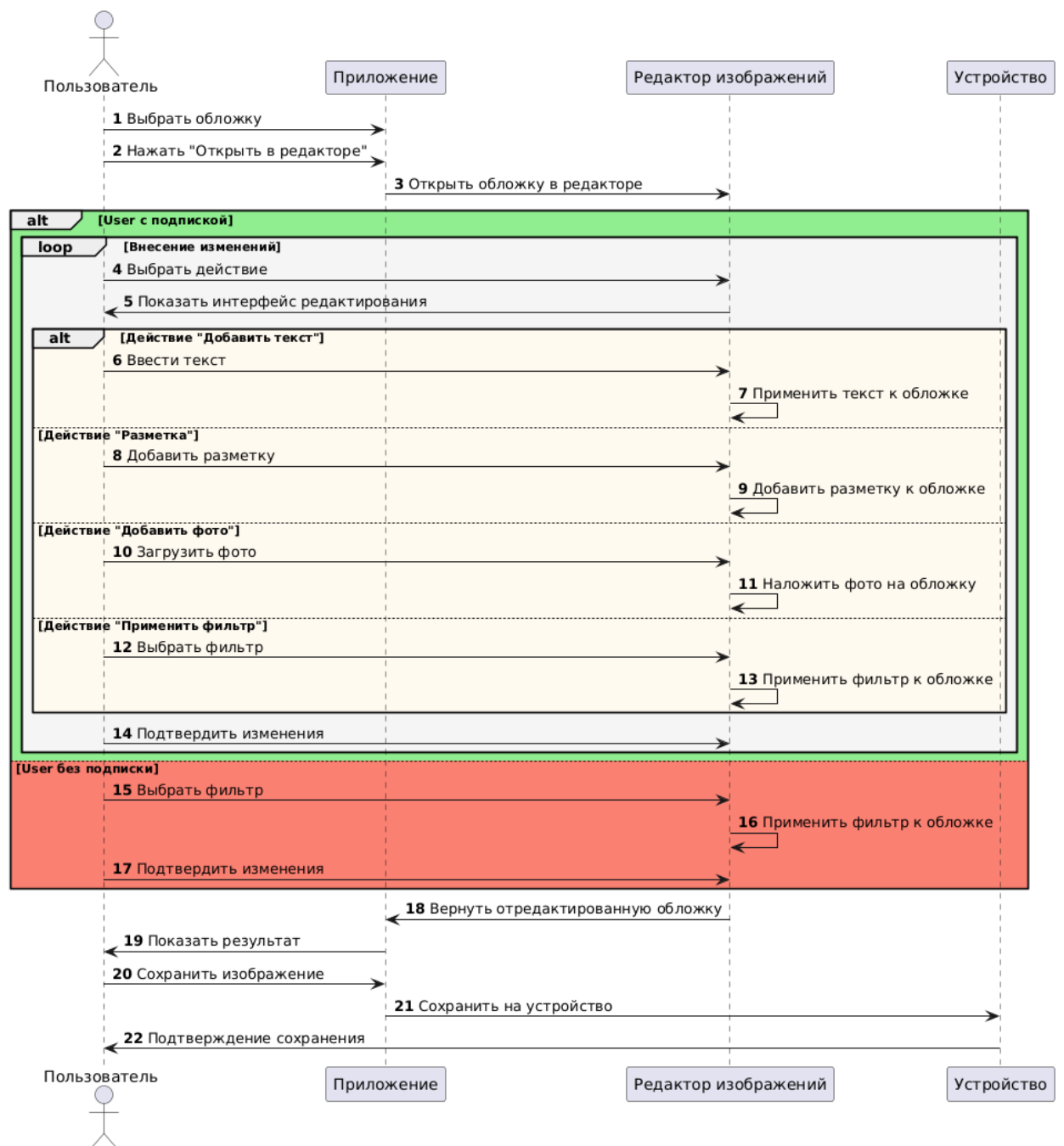


Рисунок 7 - Sequence Diagram. Редактирование обложки во встроенном редакторе изображений

2.3.3 Диаграмма состояний

Диаграмма состояний (Statechart Diagram) определяет последовательность состояний объекта, вызванных последовательностью событий. На Рисунках 8-9 рассматриваются состояния при генерации обложки и при ее редактировании.

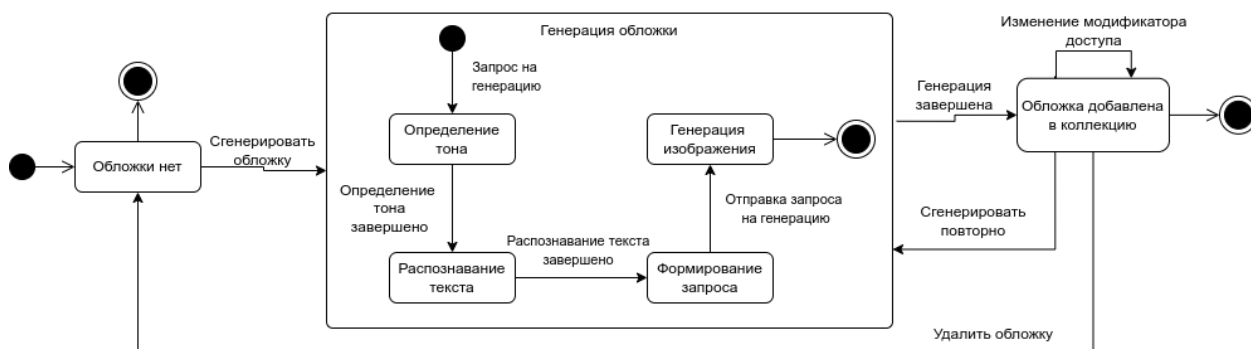


Рисунок 8 - Statechart Diagram. Генерация обложки

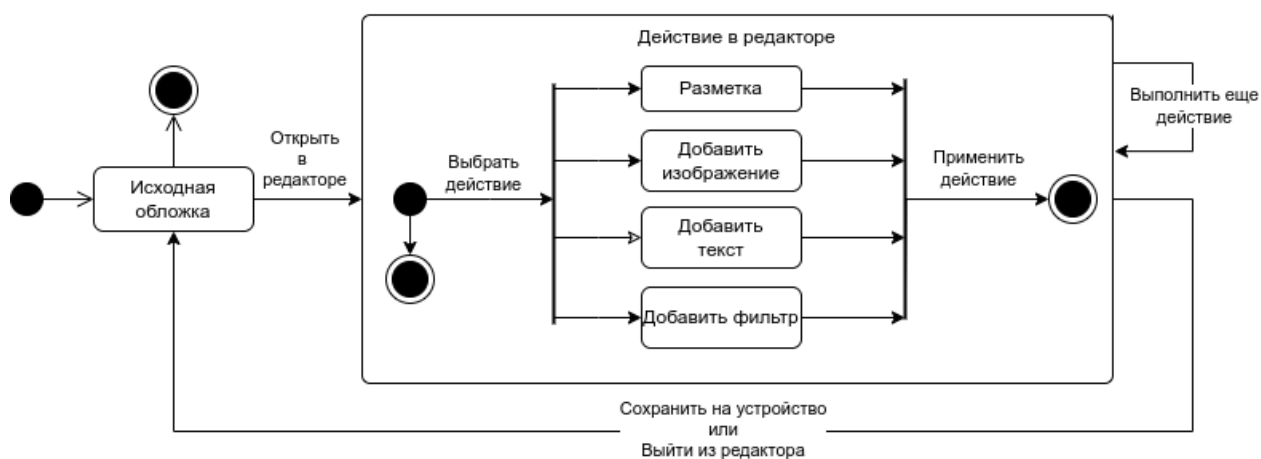


Рисунок 9 - Statechart Diagram. Редактирование обложки

2.3.4 Диаграмма активностей

Диаграмма деятельности (Activity Diagram) представляет собой диаграмму, на которой показаны действия, состояния которых описаны на диаграмме состояний. На Рисунках 10-11 рассмотрены генерация обложки и редактирование изображения.

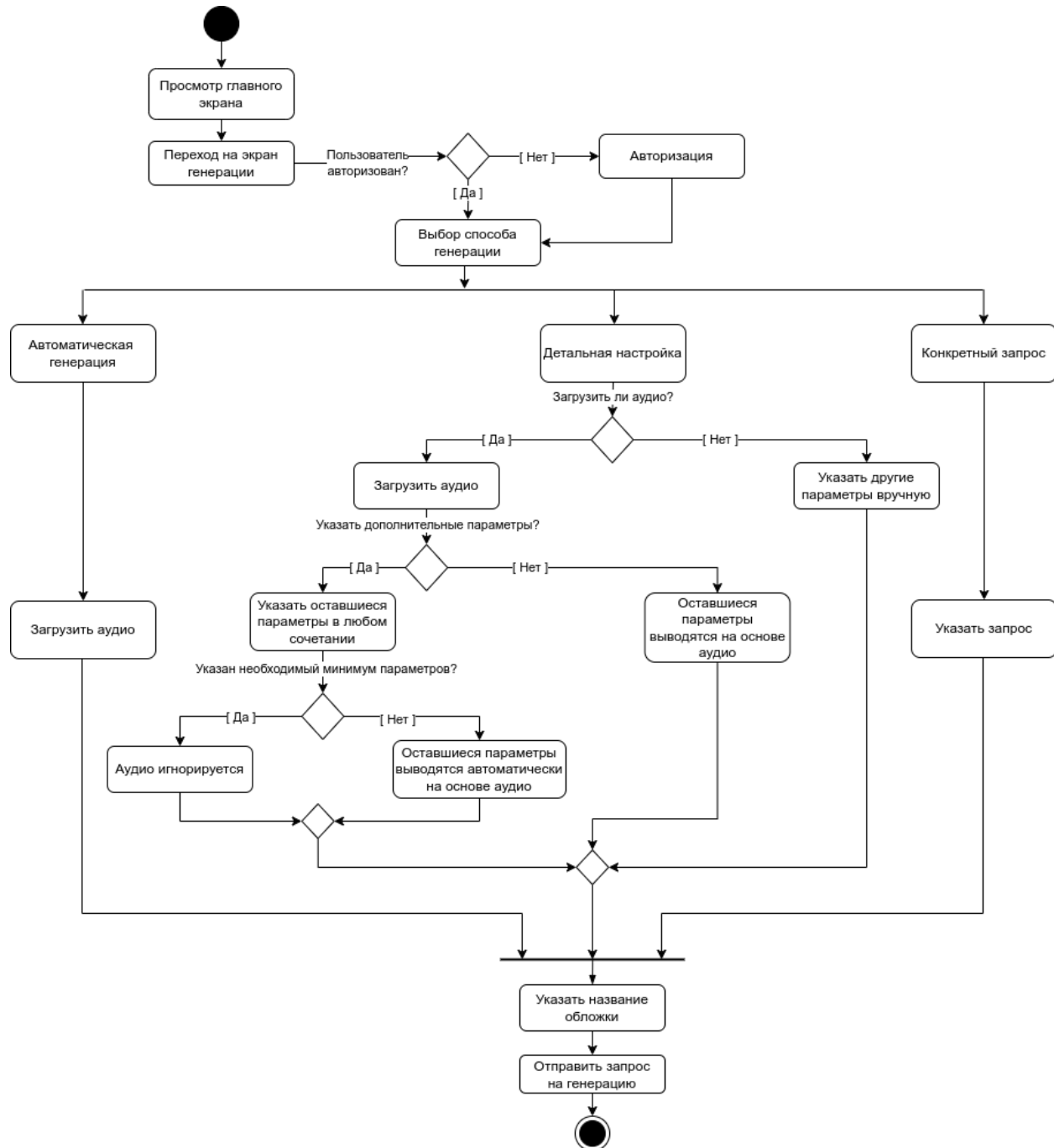


Рисунок 10 - Activity Diagram. Генерация обложки

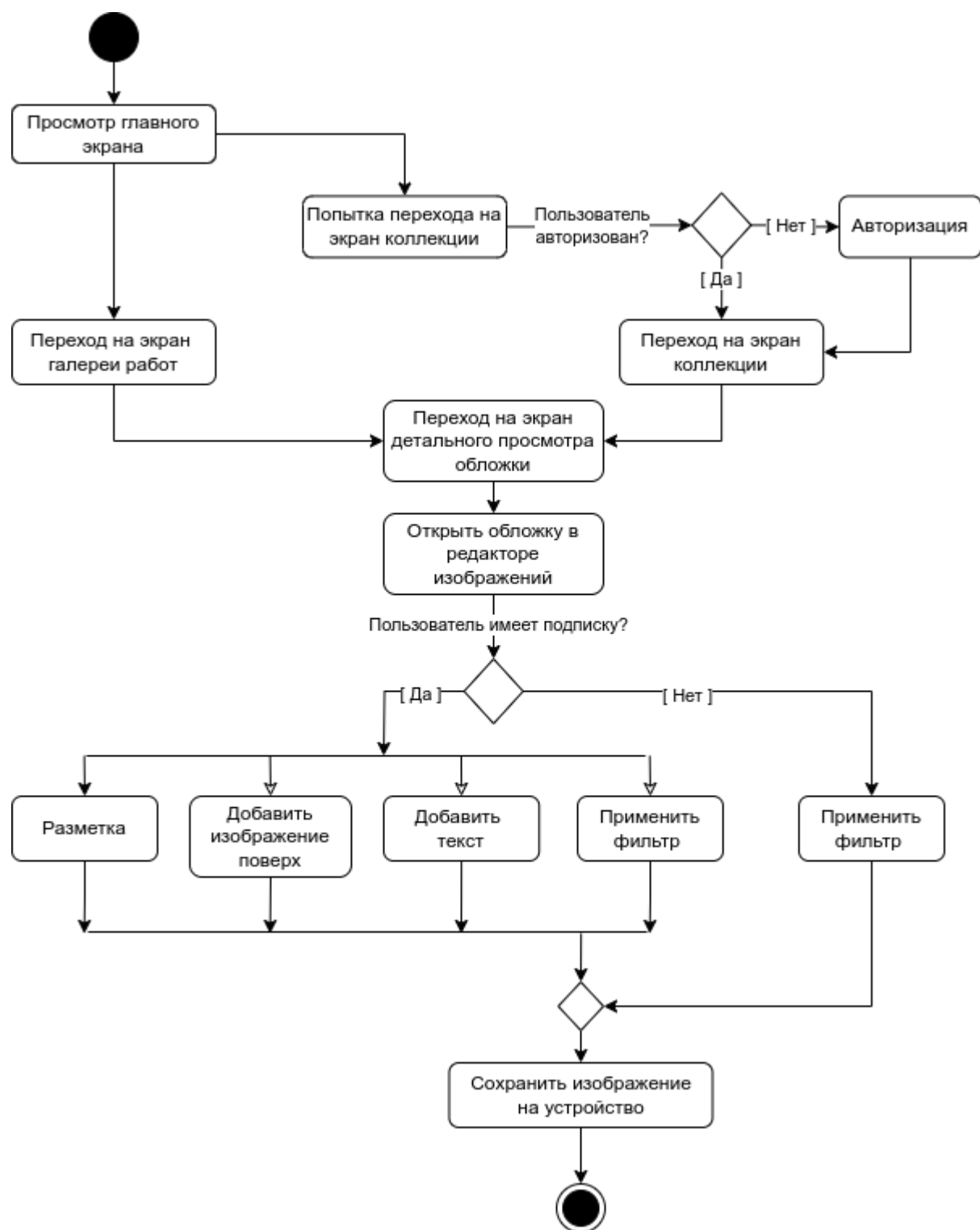


Рисунок 11 - Activity Diagram. Редактирование изображения во встроенном редакторе изображений

2.3.5 ER-диаграмма

ER-диаграмма (ER Diagram) – это модель данных, визуализирующая связь между «сущностями» внутри системы.

На Рисунке 12 представлена ER-диаграмма для разрабатываемого приложения VisualMusic.

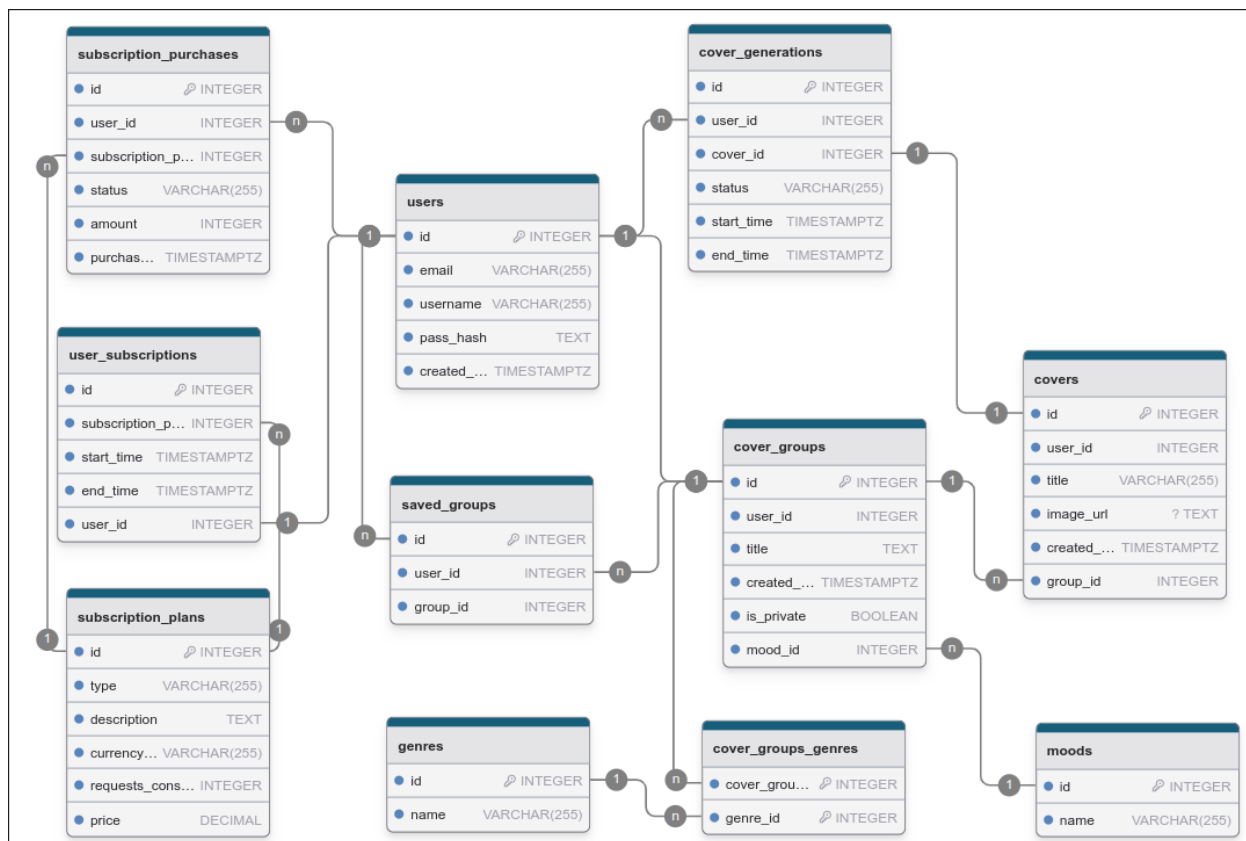


Рисунок 12 - ER Diagram

2.3.6 Диаграмма развертывания

Диаграммы развертывания (Deployment Diagram) обычно используются для визуализации физического аппаратного и программного обеспечения системы. Она моделирует физическое развертывание артефактов на узлах.

На Рисунке 13 представлена диаграмма развертывания для разрабатываемого приложения VisualMusic.

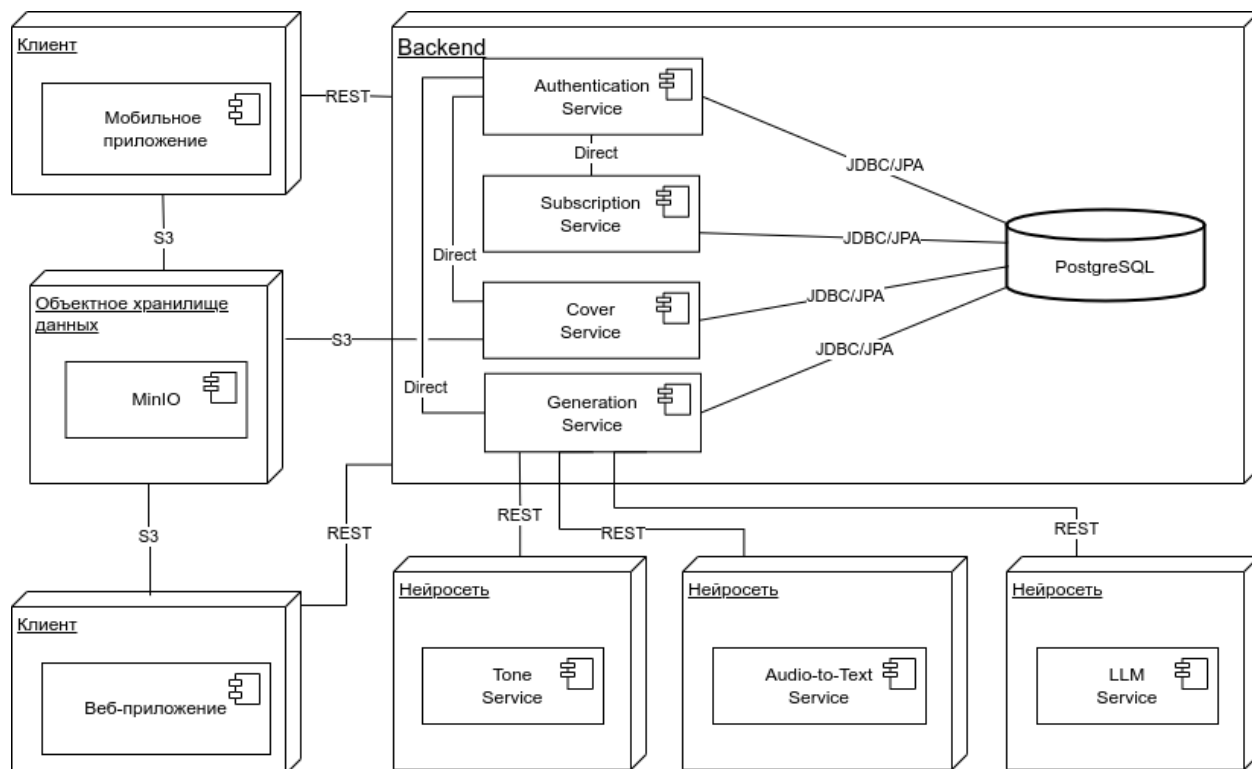


Рисунок 13 - Deployment Diagram

3 Реализация

3.1 Средства реализации

3.1.1 Средства реализации серверной части экрана

Для разработки серверной (backend) части системы был выбран следующий стек технологий:

- Spring 3.4.3 — современный высокоуровневый фреймворк для языка Java, обеспечивающий удобную реализацию REST API, поддержку безопасности, а также работу с базами данных и микросервисной архитектурой;
- PostgreSQL — это объектно-реляционная система управления базами данных (СУБД), предоставляющая мощные средства хранения, обработки и анализа структурированных данных;
- MinIO — высокопроизводительное объектное хранилище, совместимое с протоколом Amazon S3, используется для хранения медиафайлов (аудио, изображений);
- Redis — система управления данными в памяти, применяемая для кэширования и ускорения работы сервисов;
- Swagger — инструмент документирования и тестирования API, который предоставляет удобный графический интерфейс для взаимодействия с конечными точками REST-сервиса.

3.1.2 Средства реализации клиентской части сайта

HTML (HyperText Markup Language) — язык разметки, применяемый для построения структуры веб-страниц;

CSS (Cascading Style Sheets) — язык стилей, который отвечает за визуальное оформление элементов интерфейса;

Python 3.1.3 — язык программирования, используемый для создания интерактивных компонентов сайта;

Django 5.1.7 — используется для генерации HTML-страниц, обработки форм и маршрутизации, а также как платформа для административной панели;

Kotlin 2.1.0 (для Android-клиента) — современный язык программирования, применяемый для разработки клиентского интерфейса на мобильных устройствах под Android.

3.1.3 Средства реализации нейросетевой части сервиса

Для разработки нейросетевого модуля, обеспечивающего интеллектуальную обработку аудио и текста, используется следующий стек:

- Python 3.13 — основной язык разработки нейросетевых моделей;
- Django 5.1.7 — фреймворк для организации REST API и сервисной архитектуры;
- Whisper — модель от OpenAI для точного распознавания речи из аудио;
- YAMNet и SentenceTransformer — нейросетевые модели для анализа аудиоконтента и определения жанра и настроения композиции;
- YandexGPT — используется для генерации текстового описания и визуального промпта на основе входных параметров (жанра, текста песни, настроения);
- Docker — контейнеризация всех компонентов проекта для удобства развёртывания и масштабирования.

3.2 Реализация серверной части приложения

Серверная (backend) часть приложения была реализована на языке Java с использованием фреймворка Spring 3.4.3.

Структура проекта включает в себя корневой модуль и несколько ключевых подпакетов: controllers, services, models, repositories, config и utils.

В корневой папке основное внимание уделено файлу `application.yml` — он содержит все основные параметры конфигурации для запуска и корректной работы сервера.

Среди ключевых настроек:

- `spring.datasource` — параметры подключения к базе данных PostgreSQL, включая URL, имя пользователя и пароль;
- `spring.jpa` — настройки JPA/Hibernate для автоматического управления схемой базы данных;
- `server.port` — порт, на котором запускается приложение;
- `minio` — конфигурация для подключения к объектному хранилищу MinIO;
- `redis` — параметры подключения к Redis для кэширования.

Основные компоненты Spring-приложения (на примере `booking`):

- `BookingController` — REST-контроллер, принимающий HTTP-запросы и возвращающий ответы. Здесь определены конечные точки API;
- `BookingService` — бизнес-логика приложения. Здесь осуществляется обработка данных, проверка, вызов внешних сервисов и т. п.;
- `BookingRepository` — интерфейс для работы с базой данных, использующий Spring Data JPA;
- `Booking` (модель) — класс-сущность, аннотированный `@Entity`, описывающий таблицу базы данных;
- `BookingDTO` — объект передачи данных, используемый для сериализации/десериализации входных и выходных данных;

- BookingMapper — вспомогательный класс для преобразования между сущностями и DTO;
- application.yml — файл конфигурации, содержащий параметры базы данных, Redis, MinIO и других внешних систем;
- SwaggerConfig — конфигурация Swagger для генерации интерактивной документации по API.

Серверная часть приложения развёрнута с помощью Docker в изолированном контейнере, что позволяет упростить развертывание и масштабирование. Также используется Redis для хранения кэша и ускорения отклика.

Проект управляется через систему контроля версий Git, а переменные окружения (например, ключи доступа, логины и пароли) настроены через .env-файл, который передаётся контейнеру при запуске.

Для описания и тестирования REST API используется Swagger (OpenAPI). В проект добавлена зависимость springdoc-openapi-ui, которая автоматически генерирует документацию по аннотациям в контроллерах.

3.3 Реализация админ-панели

Далее следует описание структуры админ-панели VisualMusic, реализующей пользовательский интерфейс.

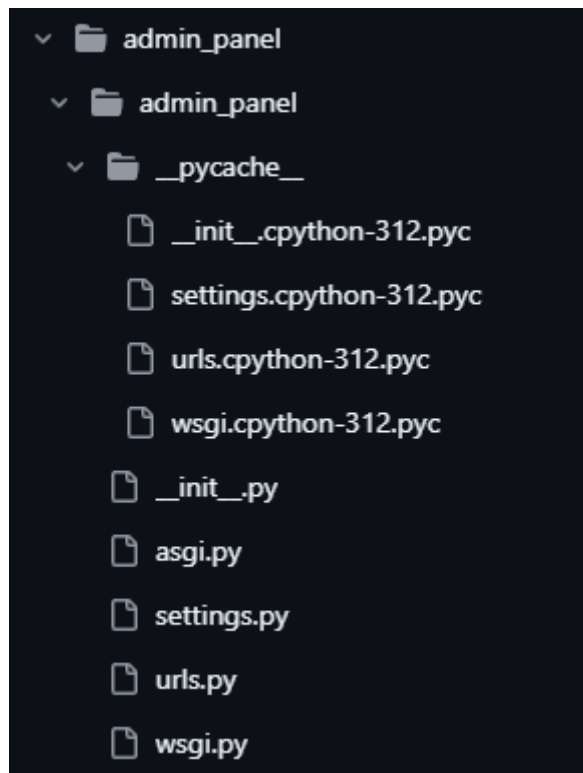


Рисунок 14 - Основной модуль админ-панели

Основной модуль проекта. Здесь находятся настройки и конфигурации всего проекта.

- `__init__.py` — файл инициализации Python-пакета.
- `settings.py` — файл настроек проекта (база данных, статические файлы, приложения и т. д.).
- `urls.py` — маршруты проекта, связывающие URL с логикой (views).
- `wsgi.py` — точка входа для WSGI-сервера.
- `asgi.py` — аналогично `wsgi.py`, но для ASGI.

Далее следует папка `static`, где содержатся статические файлы проекта — CSS и JavaScript, которые используются для стилизации и поведения страниц.

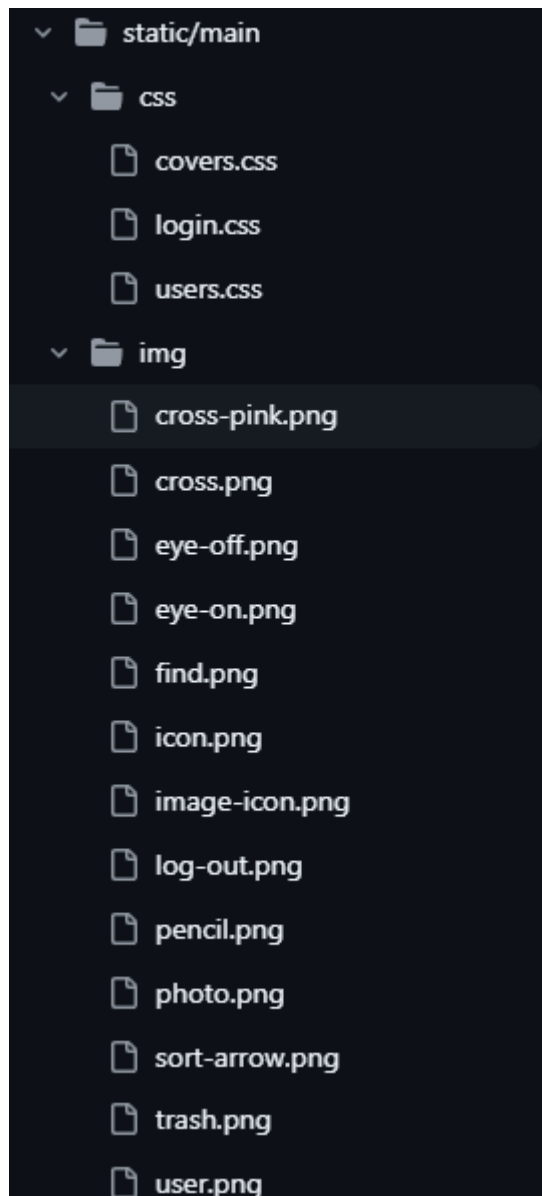


Рисунок 15 - Статические файлы проекта

CSS:

- covers.css — стилизация страницы с обложками.
- login.css — стилизация страницы входа.

js:

- covers-script.js — логика интерфейса для страницы с обложками (например, кнопки, фильтры).
- login-script.js — поведение страницы входа (валидация и пр.).

- users-script.js — взаимодействие со страницей пользователей (например, блокировка/удаление и пр.).

Следом идёт папка templates/main, HTML-шаблоны, которые отображаются пользователю.

- login.html — страница входа в админ-панель.
- users.html — интерфейс управления пользователями (просмотр, изменение, удаление).
- covers.html — отображение и модерация сгенерированных обложек.

Из основных Python-файлов присутствуют:

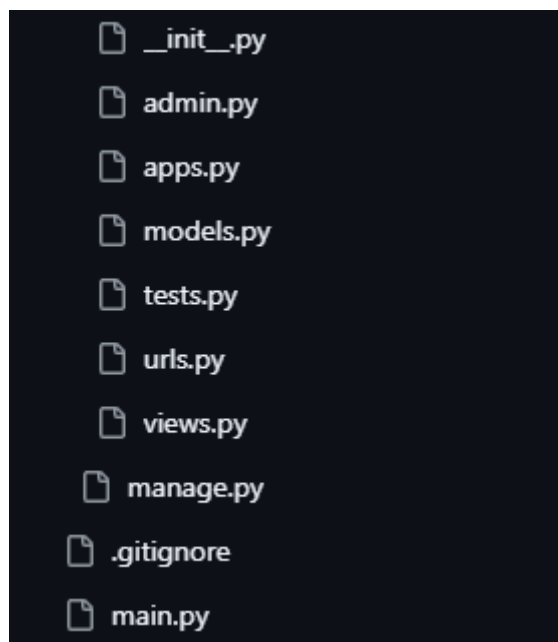


Рисунок 16 - Основные Python-файлы

- admin.py — подключение моделей к Admin.
- apps.py — конфигурация приложения.
- models.py — определение структур данных (например, пользователь, обложка и пр.).

- `views.py` — обработчики запросов (например, отображение шаблонов, возврат данных).
- `urls.py` — маршруты конкретно для приложения `main`.
- `tests.py` — модульное тестирование.
- `manage.py` — основной исполняемый файл, запускающий команды.

3.4 Реализация нейросетевых модулей

Модуль распознавания речи был реализован на языке Python с использованием фреймворка Django. Структура проекта включает в себя корневую директорию `audio-to-text-nn` и вложенное Django-приложение, где размещена логика обработки аудиофайлов (Рисунок 17).

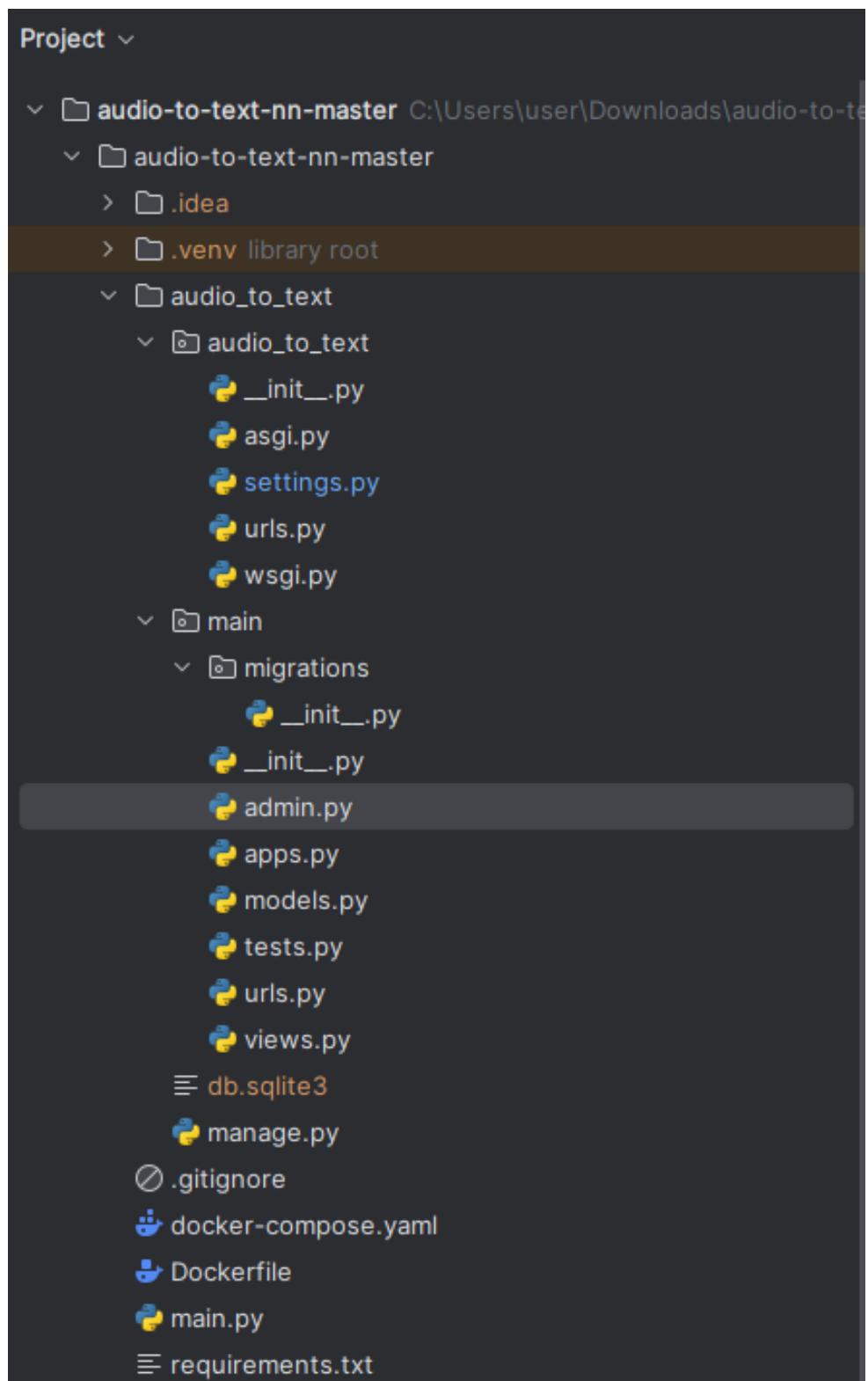


Рисунок 17 - Структура audio-to-text-nn

В папке `audio_to_text` расположены основные настройки проекта:

- settings.py — конфигурационный файл, содержащий параметры проекта,
- urls.py — маршрутная система, перенаправляющая запросы к приложениям;
- asgi.py и wsgi.py — конфигурационные файлы для запуска проекта через ASGI или WSGI-сервер.

Все ключевые компоненты функционала по распознаванию аудио находятся в main:

- models.py — определяет структуру хранения аудиофайлов и результатов распознавания;
- views.py — содержит логику обработки POST-запросов, включая: прием и сохранение аудиофайлов; запуск нейросети (например, Whisper) для распознавания речи; возврат результата в формате JSON;
- urls.py — маршруты, связывающие URL-адреса с представлениями (views);
- admin.py — регистрация моделей в административной панели;
- apps.py — конфигурация Django-приложения;
- tests.py — содержит базовые тесты для проверки API;
- migrations/ — директория с миграциями базы данных.

Для управления проектом используется файл manage.py, стандартный для Django. Также в корне проекта расположены:

- Dockerfile — инструкция для создания Docker-образа приложения;

- `docker-compose.yaml` — файл для автоматизированного запуска сервисов (если требуется подключение внешней базы данных, Redis и т.д.);
- `requirements.txt` — список всех зависимостей проекта, включая Django — основной веб-фреймворк, `torch`, `transformers`, `whisper` и др. — библиотеки, необходимые для запуска нейросети;

Распознавание речи осуществляется внутри `views.py`, где при получении аудиофайла происходит его передача в нейросетевую модель (например, Whisper от OpenAI), которая возвращает текст.

Модуль анализа жанра и эмоциональной окраски аудио реализован как отдельное Django-приложение. Оно принимает аудиофайлы и с помощью нейросетевых моделей определяет основные жанровые и тональные характеристики аудиозаписи. Архитектура проекта представлена на Рисунке 18.

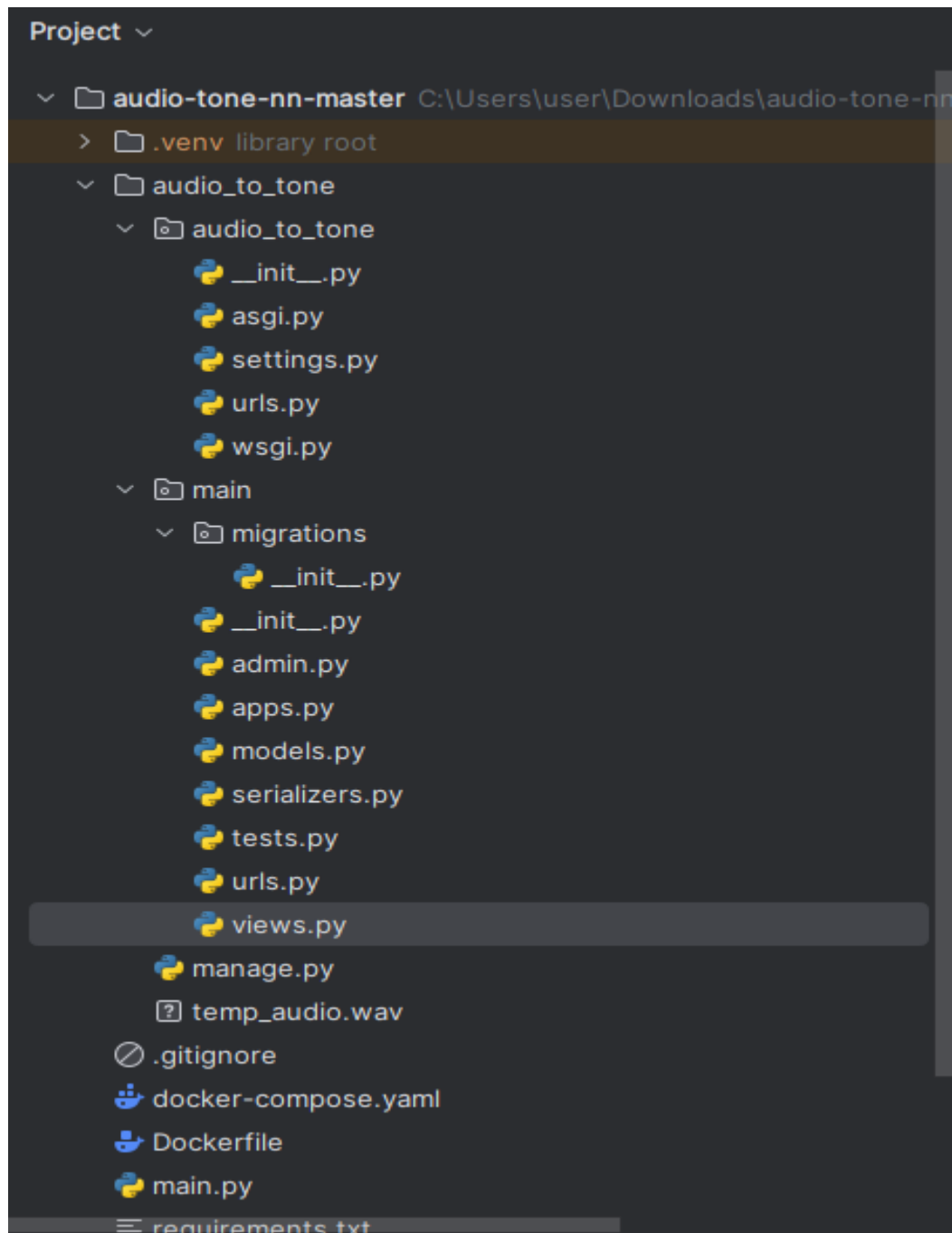


Рисунок 18 - Структура audio-tone-nn

Корневой пакет `audio_to_tone` содержит стандартные файлы:

- `settings.py` — содержит ключевые параметры:
- `urls.py` — подключение маршрутов к приложению
- `asgi.py`, `wsgi.py` — файлы запуска проекта под соответствующими серверами.

Папка `main` включает в себя реализацию REST API и взаимодействие с нейросетью, анализирующей аудио по нескольким признакам (жанр, настроение).

Ключевые модули:

- `models.py` — определяет модель базы данных для хранения загруженного файла и результатов анализа;
- `views.py` — содержит основной REST-контроллер для: приема аудиофайлов; обработки файла через нейросети; возврата JSON-ответа с жанрами и эмоциями;
- `serializers.py` — определяет сериализацию данных моделей (используется Django REST Framework);
- `urls.py` — маршруты к API;
- `tests.py` — базовые тесты API;
- `admin.py` — регистрация моделей в административной панели;
- `apps.py` — базовая конфигурация Django-приложения;
- `migrations/` — миграции базы данных.

Внутри `views.py` реализовано подключение к нейросетям:

- YAMNet (YouTube Audio Model) — модель для классификации аудиосигналов по жанрам;
- SentenceTransformer — используется для анализа эмоций, извлекаемых из текстового описания аудио или его признаков;
- Результат — список вероятных жанров, и настроение
- Другие файлы данного проекта:
- Dockerfile — инструкции для сборки образа;

- docker-compose.yaml — запуск проекта с зависимостями;
- requirements.txt — зависимости, включая: Django, torch, tensorflow, librosa, transformers; sentence-transformers, scikit-learn, numpy.

Модуль генерации текста (промпта) на основе входных характеристик — текста песни, жанров и эмоционального окраса — реализован в виде Django-приложения. Он использует предобученную языковую модель (LLM), такую как YandexGPT, для генерации промптов, которые затем используются для визуализации.

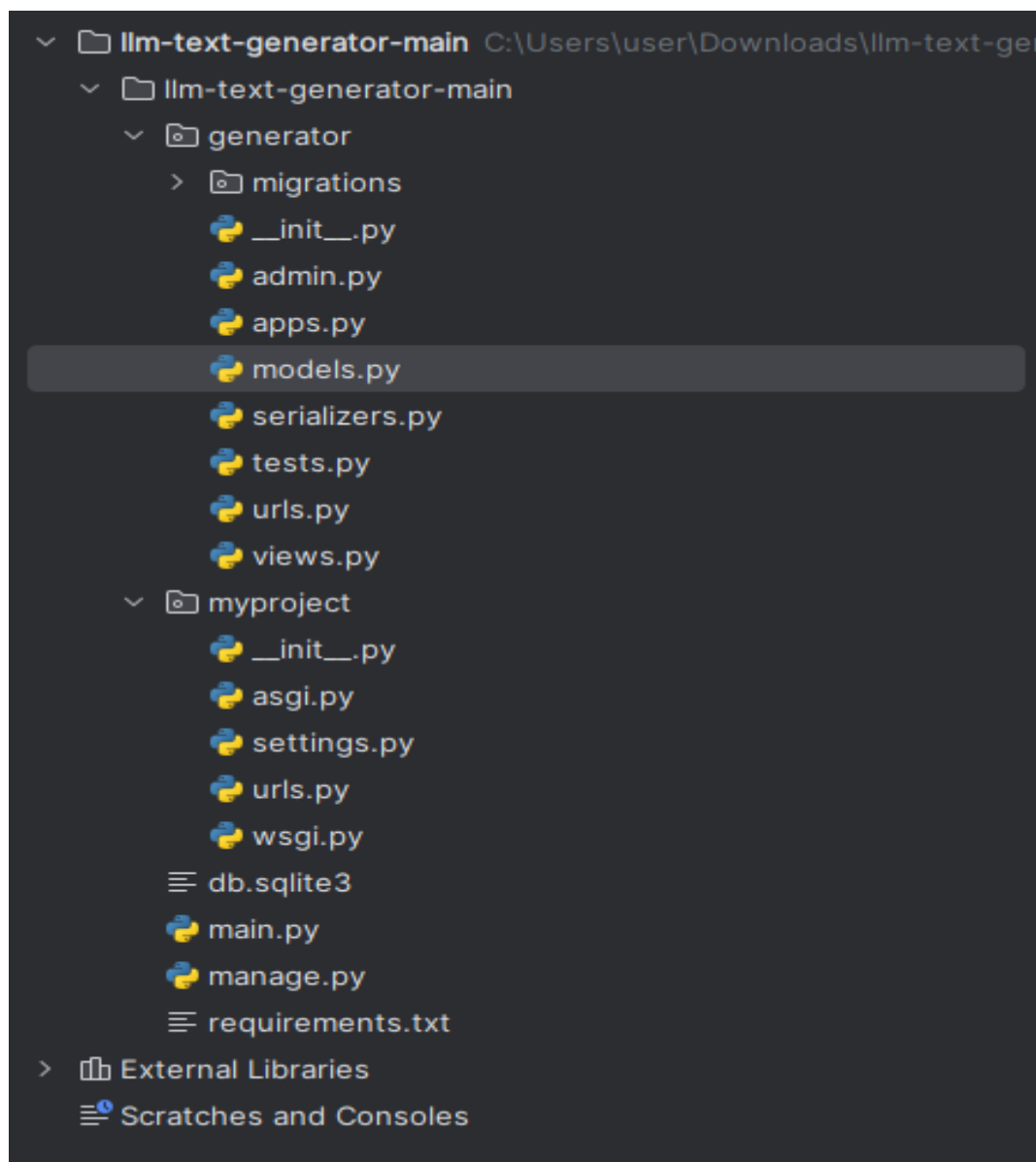


Рисунок 19 - Структура llm-text-generator

Каталог `myproject/` содержит базовые файлы:

- `settings.py` — определяет конфигурацию
- `urls.py` — подключает маршруты из `generator/urls.py`;
- `asgi.py`, `wsgi.py` — конфигурация запуска

Папка `generator` реализует всю бизнес-логику, связанную с генерацией текстов (промптов), включая: обработку входных данных (текст песни, жанры, эмоции); генерацию осмысленного описания или визуального запроса (prompt) с использованием LLM; возврат результата в виде JSON.

Ключевые файлы:

- `models.py` — содержит основную логику генерации текста. Реализует функцию взаимодействия с LLM
- `views.py` — реализует REST API: принимает POST-запрос с параметрами `text`, `genres`, `moods`; вызывает функцию из `models.py` для генерации описания; возвращает результат.
- `serializers.py` — валидирует входные данные;
- `urls.py` — маршруты для вызова API;
- `tests.py` — тестирование генерации промпта;
- `admin.py`, `apps.py`, `migrations/` — стандартные для Django файлы.

Модуль `llm-text-generator` позволяет автоматически создавать описания (промпты) для генерации изображений или обложек песен. Он опирается на входной текст, жанровую принадлежность и эмоциональное восприятие трека, обеспечивая высокую степень персонализации при визуализации музыкального контента.

3.5 Клиентская часть приложения

3.5.1 Неавторизованный пользователь

При первом открытии приложения пользователь попадает на экран входа или регистрации. Интерфейс предлагает ввести логин и пароль или пройти регистрацию, если у пользователя ещё нет учётной записи.

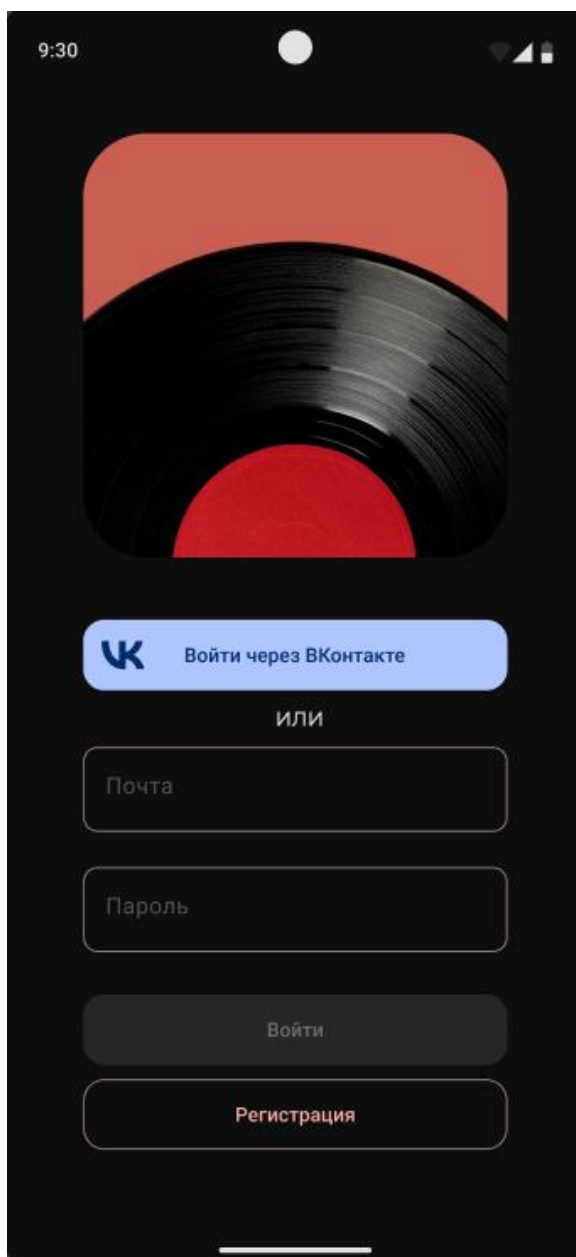


Рисунок 20 - Экран авторизации

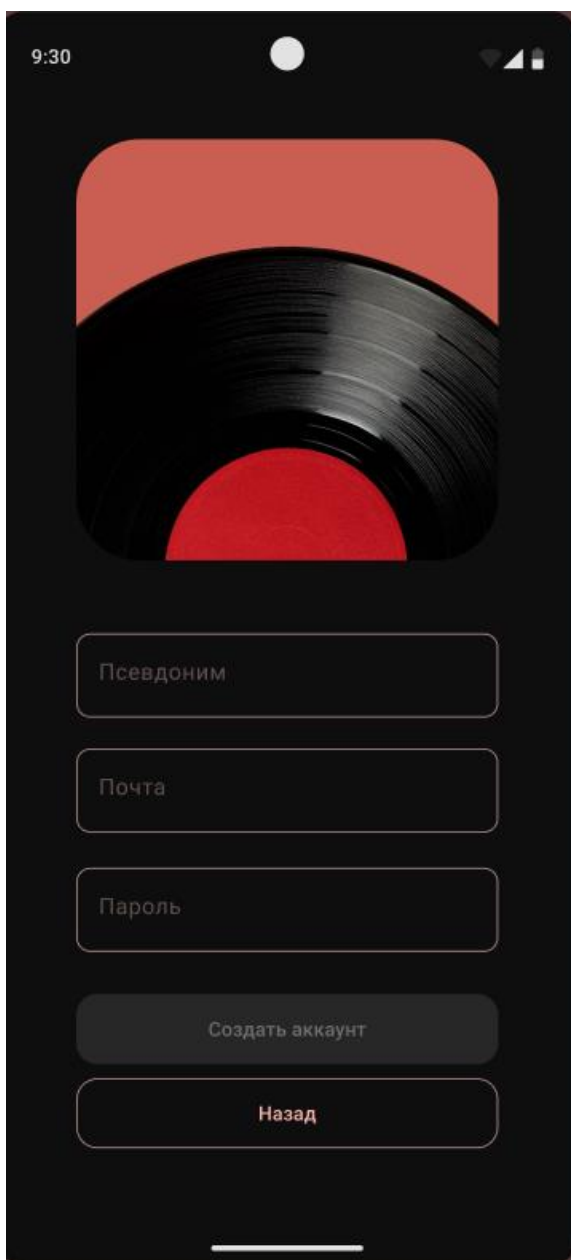


Рисунок 21 - Экран регистрации

3.5.2 Авторизованный пользователь

После входа пользователь попадает на главную страницу сервиса, где доступны основные функции:

На экране генерации доступны три режима:

- Автоматический режим — обложка генерируется на основе аудиофайла, загруженного пользователем.

- Детальный режим — пользователь также загружает аудио, но ещё может указать жанр, настроение, стиль и текст.
- Режим конкретного запроса — пользователь пишет текстом что конкретно он хочет видеть на своей сгенерированной обложке.

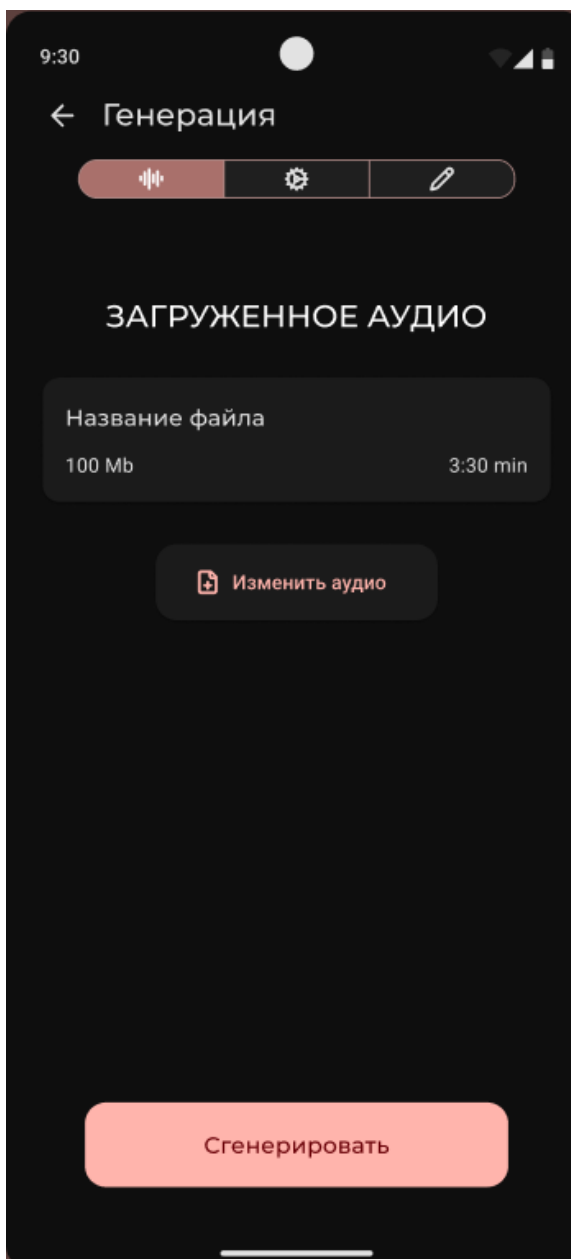


Рисунок 22 - Генерация в автоматическом режиме

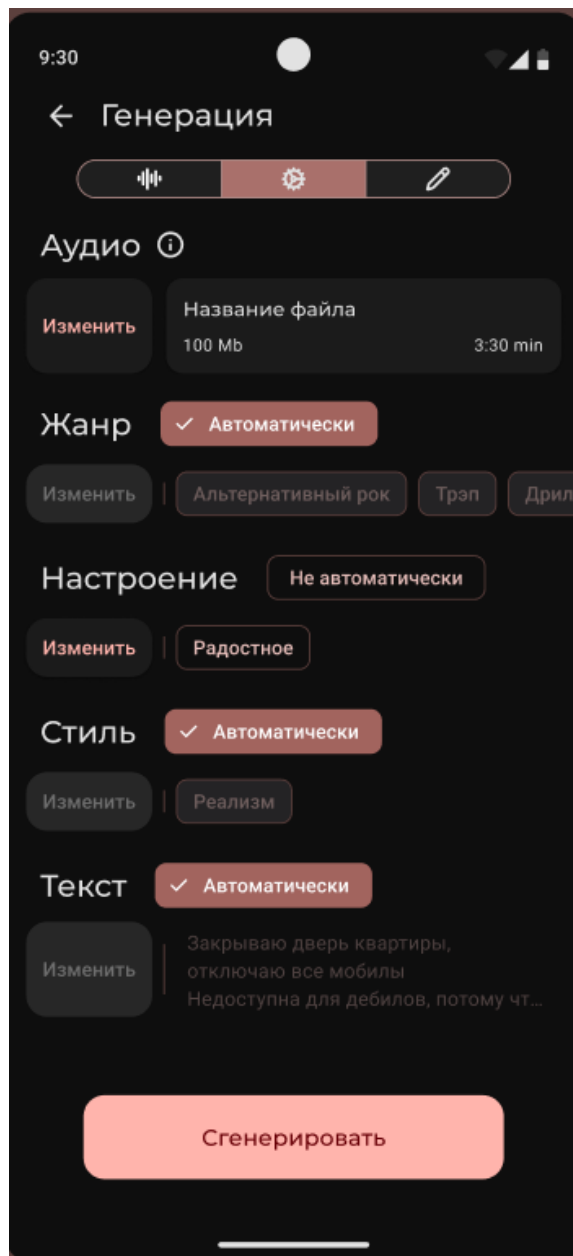


Рисунок 23 - Генерация в детальном режиме

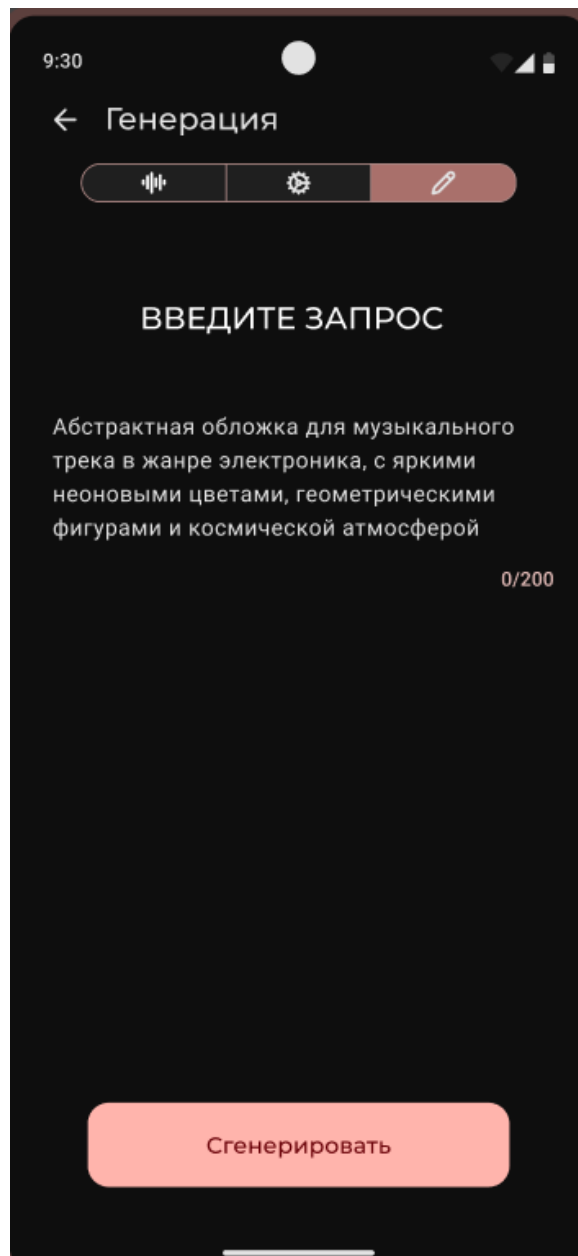


Рисунок 24 - Генерация в режиме конкретного запроса

Пользователь также может просмотреть другие обложки от пользователей, которые они сгенерировали. Каждая обложка может быть просмотрена в полном экране, скачана или добавлена в избранные. Для собственносгенерированных обложек также настроены функции приватности и удаления. Также в галереи работ можно отсортировать обложки по жанру, настроению, стилю, по дате загрузки и популярности. Эти параметры можно комбинировать.

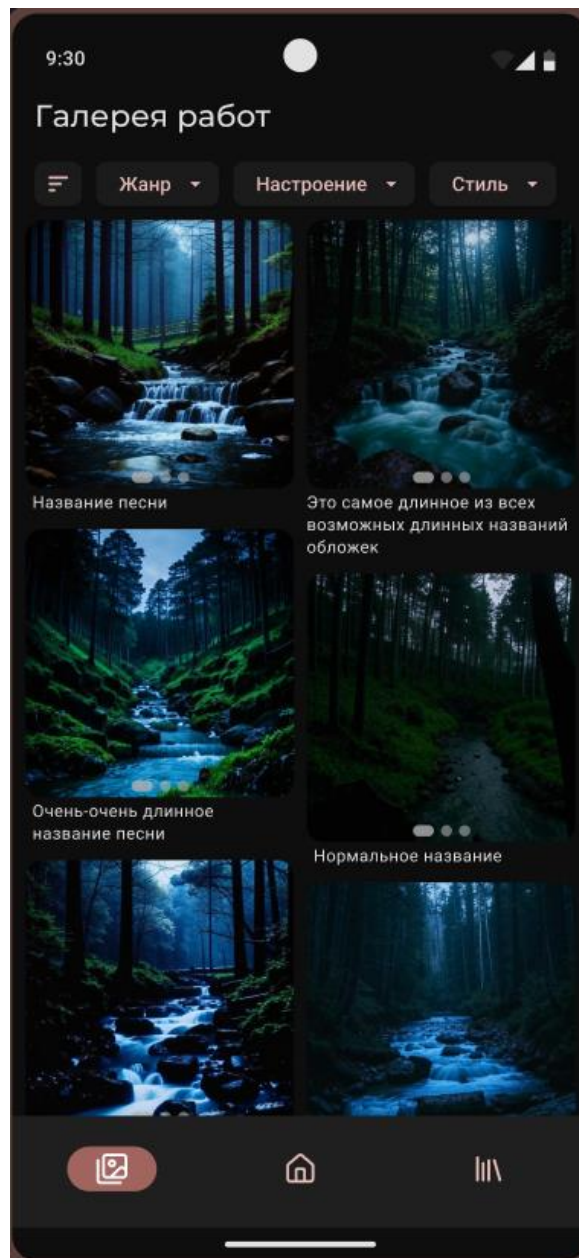


Рисунок 25 - Галерея работ

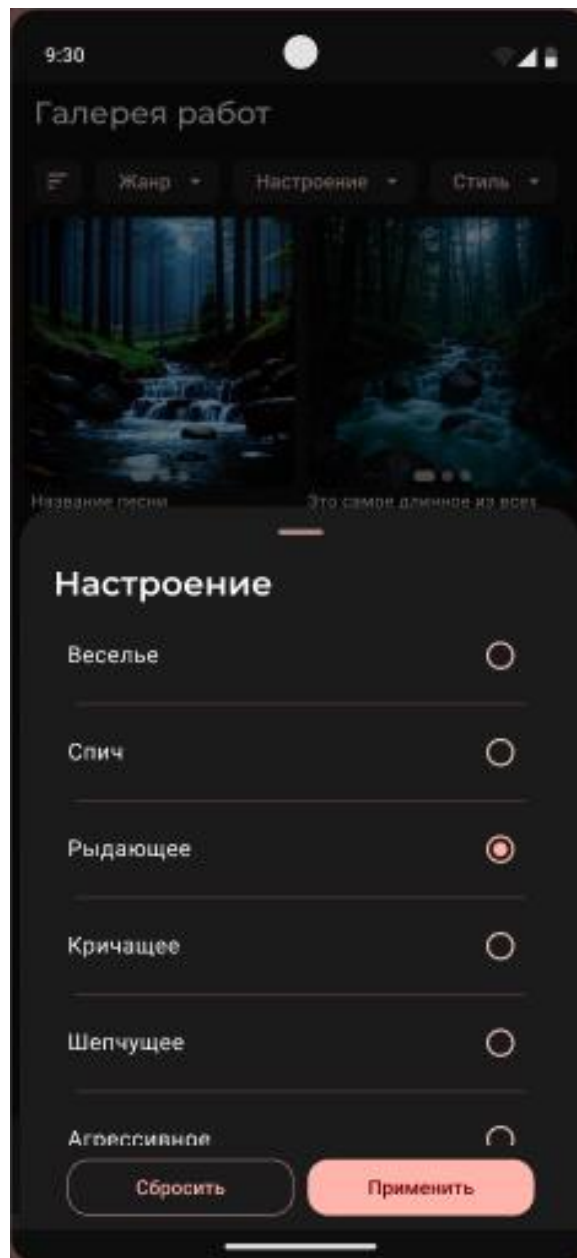


Рисунок 26 - Галерея работ. Сортировка по настроению

Заключение

В ходе выполнения курсовой работы были успешно реализованы все поставленные задачи. Был разработан интеллектуальный сервис, направленный на генерацию визуального сопровождения для музыкальных треков на основе их содержания и выразительных характеристик.

На этапе начала разработки был проведён анализ целевой аудитории и поставленных целей. Определены ключевые сценарии взаимодействия пользователя с системой, а также основные требования к качеству визуальных и смысловых результатов. По завершении разработки была проведена серия тестирований, направленных на проверку корректности работы нейросетевых моделей и взаимодействия между компонентами проекта.

Сервис решает следующие задачи для пользователей:

- Получение визуального оформления трека в автоматическом режиме;
- Интерпретация музыкального жанра и эмоционального окраса аудиофайла;
- Генерация описания или промпта, соответствующего стилю и содержанию композиции;
- Улучшение узнаваемости и эстетической привлекательности треков без необходимости ручного дизайна.

Таким образом, итоги разработки подтверждают достижение поставленных целей. Реализованный проект позволяет музыкальному контенту получить уникальное визуальное сопровождение, усиливающее эмоциональное восприятие композиции и облегчающее продвижение трека в цифровой среде.

Список использованных источников

1. Django — фреймворк для веб-разработки на Python [Электронный ресурс]. — Режим доступа: <https://docs.djangoproject.com/ru/5.0/intro/overview/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
2. Docker — контейнеризация приложений [Электронный ресурс]. — Режим доступа: <https://www.docker.com/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
3. PyTorch — платформа для глубокого обучения [Электронный ресурс]. — Режим доступа: <https://pytorch.org/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
4. TensorFlow Hub — библиотека моделей машинного обучения [Электронный ресурс]. — Режим доступа: <https://www.tensorflow.org/hub>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
5. SentenceTransformers — библиотека для семантического поиска и сравнения предложений [Электронный ресурс]. — Режим доступа: <https://www.sbert.net/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
6. YandexGPT — генерация текста на естественном языке [Электронный ресурс]. — Режим доступа: <https://huggingface.co/yandex/YandexGPT-5-Lite-8B-pretrain>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
7. FastAPI — современный веб-фреймворк для создания API [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/ru/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).

8. Swagger — документация и тестирование REST API [Электронный ресурс]. — Режим доступа: <https://swagger.io/specification/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
9. MinIO — объектное хранилище [Электронный ресурс]. — Режим доступа: <https://min.io/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).
10. PostgreSQL — система управления базами данных [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/>. — Заглавие с экрана. — (Дата обращения: 04.06.2025).